知識表現と推論

(knowledge representation and reasoning)

Unit 16　ソルバーに渡すもの (input to solvers)

宋 剛秀 (そう たけひで)

名古屋大学

この unit の内容

- **内容**：Sugar の CSP 形式と，それを生成する Syrup
- **前の unit**：Unit 15　alldifferent と組み立て方
- **次の unit**：Unit 17　三つのモデル

この unit の到達点

小さな問題を Syrup で書いて解ける．

この unit の進め方

Unit 16–18 は，スライドを読みながら，手元でモデルを書いて解く．

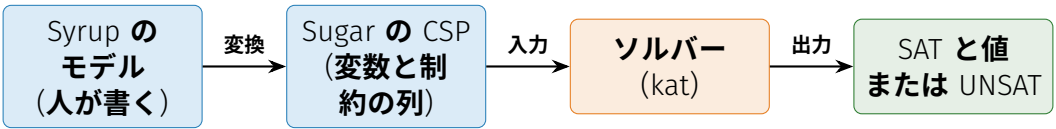
- 使うもの
  - ブラウザで動く kat-syrup sandbox を使う．インストールは要らない．
  - URL は Slack で知らせる．
- 進め方
  - スライドのモデルを，そのまま書き写して解く．
  - 数字や制約を変えて，結果がどう変わるかを確かめる．
- スライドの数字
  - スライドにあるモデルは，すべて実際に解いて確かめた．
  - 変数や制約の数は，ソルバーが出す統計の値である．

これまでの unit と，ここからの位置

- **Unit 04 から Unit 15**
  - 解法の中身を見た．二項化，伝播，探索，伝播器である．
  - 手で動かすと，小さな問題でも手間がかかった．
- **Unit 16–18**
  - 解法はソルバーに任せる．人が書くのは，問題そのものである．
  - 問題を，ソルバーが読める形に書いたものを**モデル**と呼ぶ．

モデル化は知識表現，ソルバーの仕事は推論である (Unit 02)**．**

ソルバーの入力と出力



- **Sugar の CSP 形式**：変数の宣言と制約を，S 式で並べたものである．ソルバーはこれを読む．
- **Syrup**：Sugar の CSP を，ルールでまとめて生成する言語である．
- **出力**：解があれば SAT と各変数の値，なければ UNSAT が返る．

まず Sugar の形式を見て，次に Syrup を見る．

Sugar の CSP 形式 (1)：変数と制約

一行に一つ，変数の宣言か制約を書く．どれも括弧で囲んだ S 式である．

```
(int x 1 3)
(int y 1 3)
(!= x y)
(< (+ x y) 5)
```

- `(int x 1 3)`：整数変数 x，ドメイン $[1, 3]$
- `(!= x y)`：$x \neq y$
- `(< (+ x y) 5)`：$x + y < 5$

- 演算子を先に書く．$x + y$ は `(+ x y)` と書く．
- 制約は並べた順に意味が変わらない．すべてを同時に満たす解を探す．

Sugar の CSP 形式 (2)：よく使う制約

書き方	意味
`(= a b)`, `(!= a b)`, `(< a b)`, `(<= a b)`	比較
`(+ a b c)`, `(- a b)`, `(* 3 a)`, `(abs a)`	算術
`(and p q)`, `(or p q)`, `(not p)`, `(imp p q)`	論理．`imp` は「p なら q」
`(alldifferent a b c)`	すべて違う値（グローバル制約）
`(objective minimize k)`	k を最小にする解を求める

- 比較や論理を組み合わせて，内包的な制約を書く (Unit 04)．
- `alldifferent` は，Unit 04 のグローバル制約である．その伝播は Unit 15 で見た．
- `objective` を書くと，制約最適化問題になる．

例：Unit 03 の地域の色

地域 X，Y，Z を 2 色で塗る．隣り合うのは X—Y と Y—Z である．

```
(int x 1 2)
(int y 1 2)
(int z 1 2)
(!= x y)
(!= y z)
```

- **色を整数 1, 2 で表す．**
- **隣り合う組だけに != を書く．**

ソルバーの出力

SAT．$x = 2$，$y = 1$，$z = 2$

Unit 03 **で手で導いた「X と Z は同じ色」が，解にも現れている．**

例：Unit 03 の席

三人 A，B，C を席 1〜3 に座らせる．A と B は隣，C は端である (Unit 03 の例 1)．

```
(int a 1 3)
(int b 1 3)
(int c 1 3)
(alldifferent a b c)
(= (abs (- a b)) 1)
(or (= c 1) (= c 3))
```

- **「別々の席」は** `alldifferent`**．**
- **「隣」は，席の番号の差の絶対値が 1．**
- **「端」は，1 か 3 のどちらか．** `or` **で書く．**

ソルバーの出力

SAT．$a = 3$，$b = 2$，$c = 1$

解がないとき：UNSAT

三つの地域が互いに隣り合い，色が二つなら，解はない．

```
(int x 1 2)
(int y 1 2)
(int z 1 2)
(alldifferent x y z)
```

ソルバーの出力

UNSAT

- 解がないことを，ソルバーが示した．
- Unit 08 と Unit 12 では，アーク整合では分からない例だった．

UNSAT は「探し方が悪くて見つからない」ではない．どの割当ても制約を満たさないという意味である．

手で書く限界

4 クイーンを Sugar で書くと，次のようになる．

```
(int q_1 1 4)
  ...
(int q_4 1 4)
(!= q_1 q_2)
  ...
(!= (- q_2 q_1) 1)
(!= (- q_3 q_1) 2)
  ...
```

- 4 クイーンで 22 行，8 クイーンで 92 行になる．
- どの行も，同じ形のくり返しである．
- 盤の大きさを変えると，全部を書き直すことになる．

「どの二つの行についても」という規則のまま書きたい．これが Syrup である．

Syrup：ルールで制約を生成する

Syrup は，Sugar の CSP を生成するための言語である．二つの層からなる．

- メタの層 (ASP や Datalog に似た形)
  - 問題のデータをファクトで書く．例：`edge(x,y)`.
  - ファクトから新しいファクトを導くルールを書く．
- Sugar の層 (S 式)
  - 変数の宣言と制約を，S 式で書く．
  - S 式にルールの条件を付けると，条件を満たすだけ S 式が作られる．

Syrup のモデルは，まず Sugar の CSP に変換され，それがソルバーに渡る．

Syrup の書き方 (1)：ファクトとルール

```
edge(x,y).
edge(y,z).
node(U) :- edge(U,_).
node(V) :- edge(_,V).
row(1..4).
```

- `edge(x,y).` はファクトである．
- `node(U) :- edge(U,_).` はルールである．「`edge(U,_)` なら `node(U)`」と読む．
- 大文字で始まる名前は変数，`_` は使わない引数である．
- `row(1..4).` は，`row(1)` から `row(4)` の四つのファクトである．

この例から導かれるのは，`node(x)`，`node(y)`，`node(z)` である．

Syrup の書き方 (2)：S 式を生成する

S 式の後ろに `:-` と条件を書くと，条件を満たす組ごとに S 式が一つ作られる．

```
(int c[U] 1 2) :- node(U).
(!= c[U] c[V]) :- edge(U,V).
```

- `c[U]` は，`U` の値を名前に付けた変数である．`U` が `x` なら `c_x` になる．
- 1 行目は，`node` のファクトごとに変数を宣言する．
- 2 行目は，`edge` のファクトごとに $\neq$ を書く．

条件には，比較 (`R < S`) や，範囲を回す束縛 (`P = 0..1`) も書ける．

Syrup の変換で起きること

Syrup のモデルは，二段階で Sugar の CSP になる．

1. ファクトを導く
   - ファクトとルールから，導けるファクトをすべて導く．
   - 例：`edge(x,y).` と `node(U) :- edge(U,_).` から `node(x)`．
2. S 式を作る
   - S 式のルールごとに，条件を満たす組をすべて探し，そのたびに S 式を一つ書き出す．
   - 例：`edge(x,y)` から `(!= c_x c_y)`．

- sandbox で「Include META language in Transpile output」を選ぶと，1 で導いたファクトも表示される．
- ファクトの段階は，ソルバーを使わずに計算が終わる．制約を解くのはソルバーの仕事である．

Syrup の書き方 (3)：seq で並べる

一つの制約の中に，条件を満たすものを並べたいときは `seq` **を使う．**

```
row(1..4).
(int q[R] 1 4) :- row(R).
(alldifferent (seq q[R] : row(R)))
```

- `(seq q[R] : row(R))` **は，** `q_1 q_2 q_3 q_4` **に展開される．**
- `3` **行目は，**`(alldifferent q_1 q_2 q_3 q_4)` **という一つの制約になる．**
- **並べるものは変数でなくてもよい．**`(seq (+ q[R] R) : row(R))` **のような式も並べられる．**

例：地域の色を Syrup で書く

Syrup

```
edge(x,y). edge(y,z).
node(U) :- edge(U,_).
node(V) :- edge(_,V).
(int c[U] 1 2) :- node(U).
(!= c[U] c[V]) :- edge(U,V).
```

変換された Sugar

```
(int c_x 1 2)
(int c_y 1 2)
(int c_z 1 2)
(!= c_x c_y)
(!= c_y c_z)
```

- 地図を変えるときは，edge のファクトだけを書き換える．
- 解は $c_x = 2$，$c_y = 1$，$c_z = 2$ である．前の Sugar の例と同じである．

sandbox の使い方

1. エディタに Syrup のモデルを書く．
2. 「Transpile」で，生成された Sugar の CSP を確かめる．
   - 制約が思ったとおりの数だけ作られているかを見る．
3. 「Solve」で，ソルバーの結果を見る．
   - SAT と値，または UNSAT が出る．最適化なら目的関数の値も出る．
   - 変数の数，制約の数などの統計も出る．

- sandbox には，Syrup を一歩ずつ学ぶチュートリアルもある．
- 書き方に迷ったら，チュートリアルの第 1 章（数独まで）を先に進めるとよい．